

CLASE DE REPASO

Ejes III y IV

EL ECOSISTEMA NOSQL, ARQUITECTURAS HÍBRIDAS E INTEGRACIÓN

Base de Datos III

Ecosistema NoSQL: Los 3 pilares



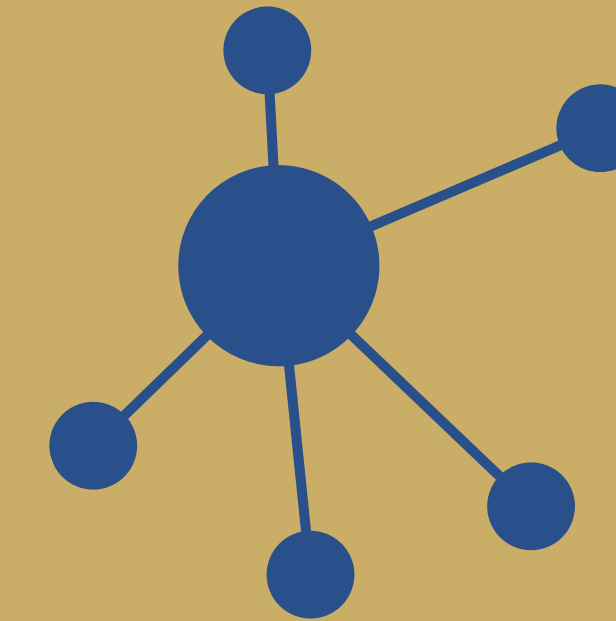
Documental (MongoDB)

Orientado a lecturas rápidas.
Uso de BSON y
desnormalización
Limite: 16MB



Clave-Valor (Redis)

Almacenamiento en
memoria (RAM).
Latencia de microsegundos.
Estructuras nativas y TTL.



De Grafos (Neo4j)

Nodos, aristas y relaciones.
Todos pueden contener
información propia.
Motor Cyper.

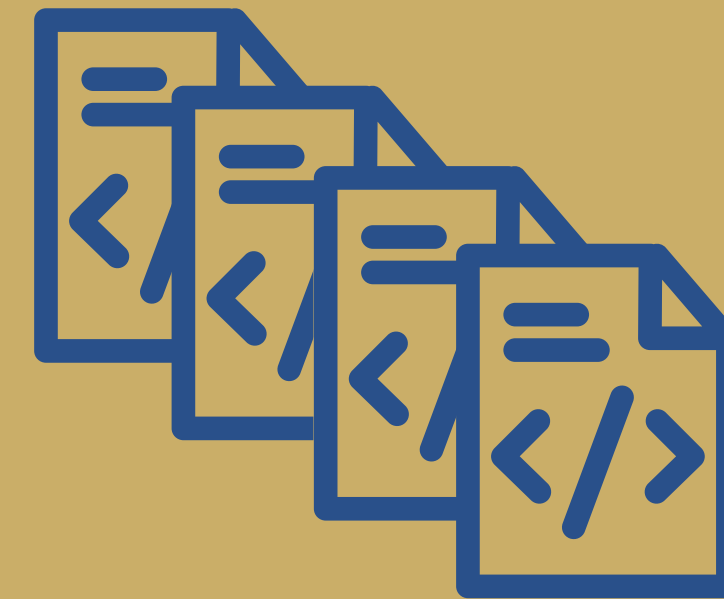
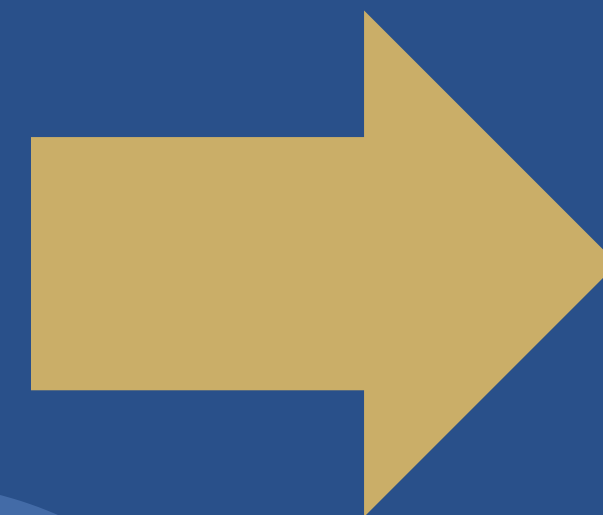
VENTAJAS:

- Ofrece modelos más flexibles que SQL.
- Diseñado para escalar de forma horizontal.
- Baja Latencia.

MongoDB



La unidad mínima de información es un documento.

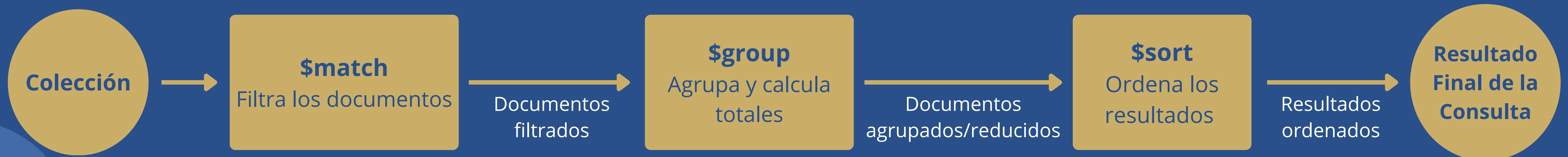


Un grupo de documentos forman una colección.

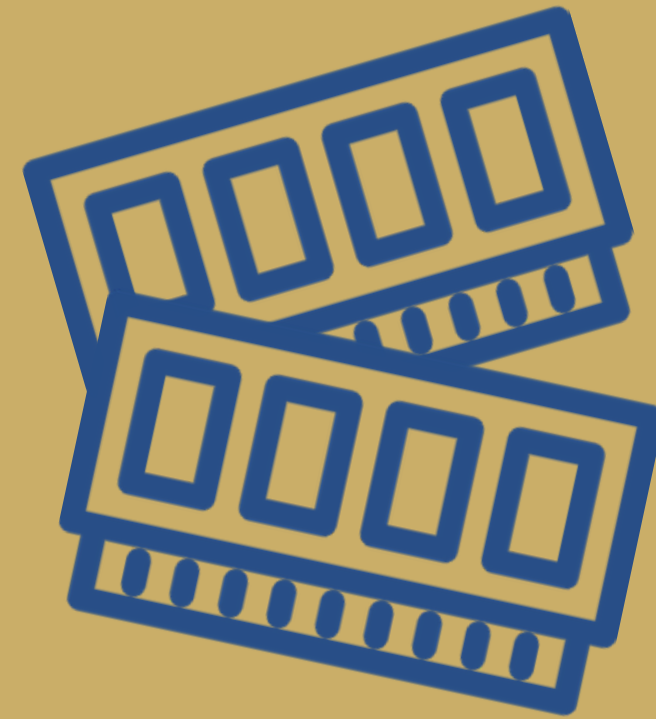
Características:

- Se almacenan datos embebidos.
- internamente el motor almacena la información en formato BSON.

STAGES EN AGGREGATION FRAMEWORK



Redis



Almacén de
estructuras
de datos en
memoria



Tiempo de
vida para
expiración
de datos



Snapshots
para copias
de seguridad
en el disco
duro

CASOS DE USO PRINCIPALES:

- **Caché:** Guarda las respuestas de consultas pesadas en una capa intermedia.
- **Gestión de Sesiones de Usuario:** Almacena variables de sesión para responder de forma instantánea.
- **Colas de Mensajería y Tareas:** implementa flujos de trabajo asíncronos y colas de tareas eficientes.
- **Análisis en Tiempo Real y Leaderboards:** calcula puntuaciones en tiempo real.
- **Patrón Pub/Sub (Publicación/Suscripción):** Permite crear aplicaciones de mensajería o chat en tiempo real

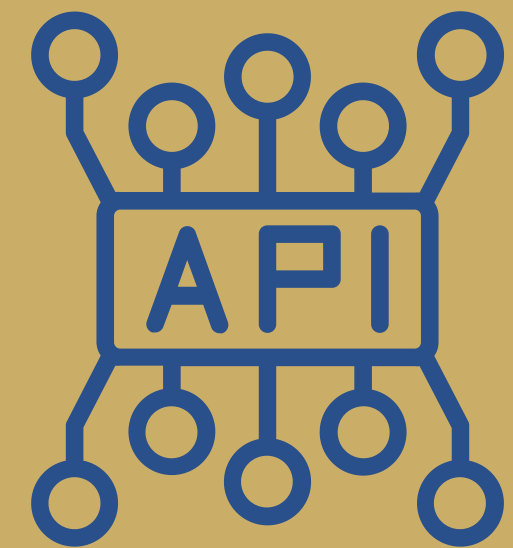
Neo4j



- Utiliza Cypher como lenguaje de consultas.
- Al crear una relación, se guarda físicamente como un puntero en la memoria (Index-free adjacency).

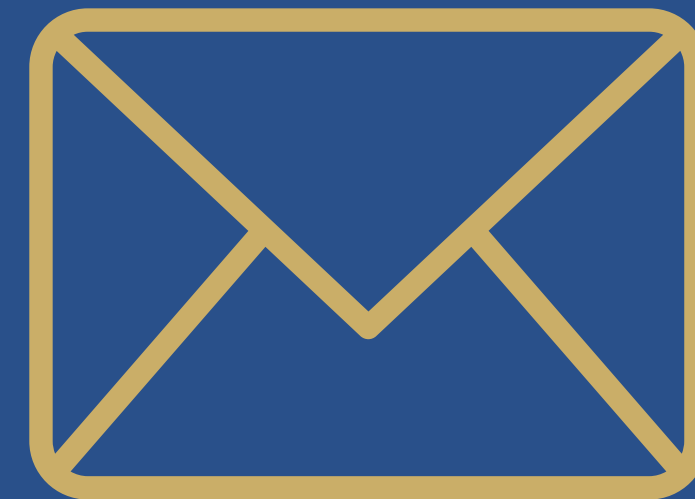
Usos recomendados	Usos NO recomendados
Motores de Recomendación	Si los datos están aislados y rara vez se cruzan
Detección de Fraudes	Si se necesita un registro transaccional financiero puro o auditorías masivas
Gestión de Redes e IT	Si se guardan datos de series temporales, como métricas de IoT por segundo
Rutas y Logística	

APIs + Postman

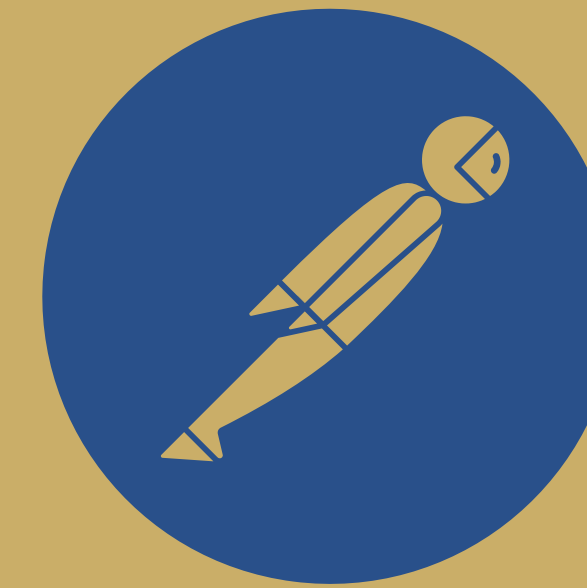


Interfaces de Programación de Aplicaciones

conjunto de reglas y protocolos que permite que dos aplicaciones de software se comuniquen y compartan datos entre sí



Petición HTTP



PostMan

permite crear peticiones HTTP, para enviarlas a nuestra API, y analizar la respuesta

MÉTODOS HTTP

POST → Create

GET → Read

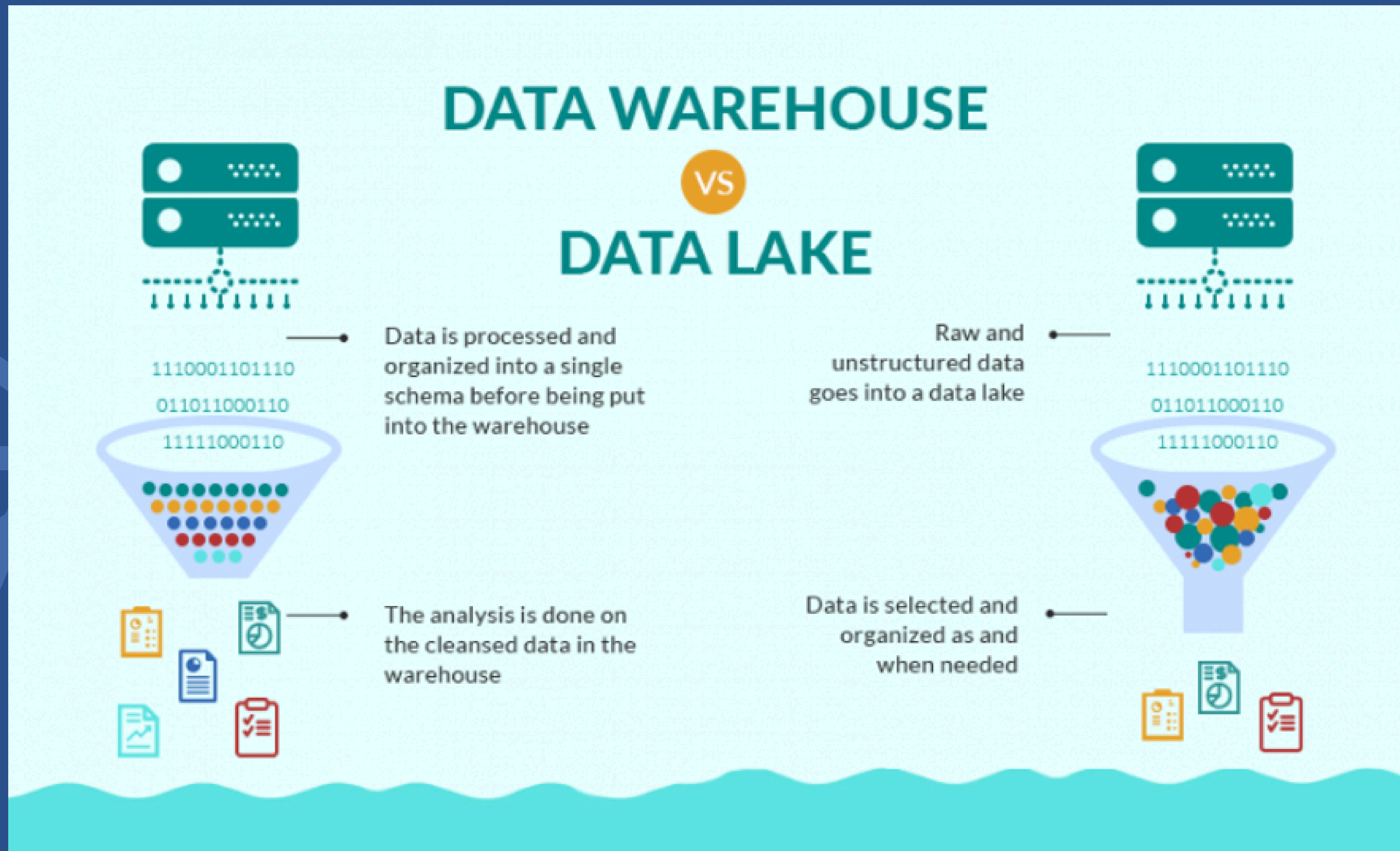
PUT / PATCH → Update

DELETE → Delete

ABSTRACCIÓN DE LA CAPA DE DATOS

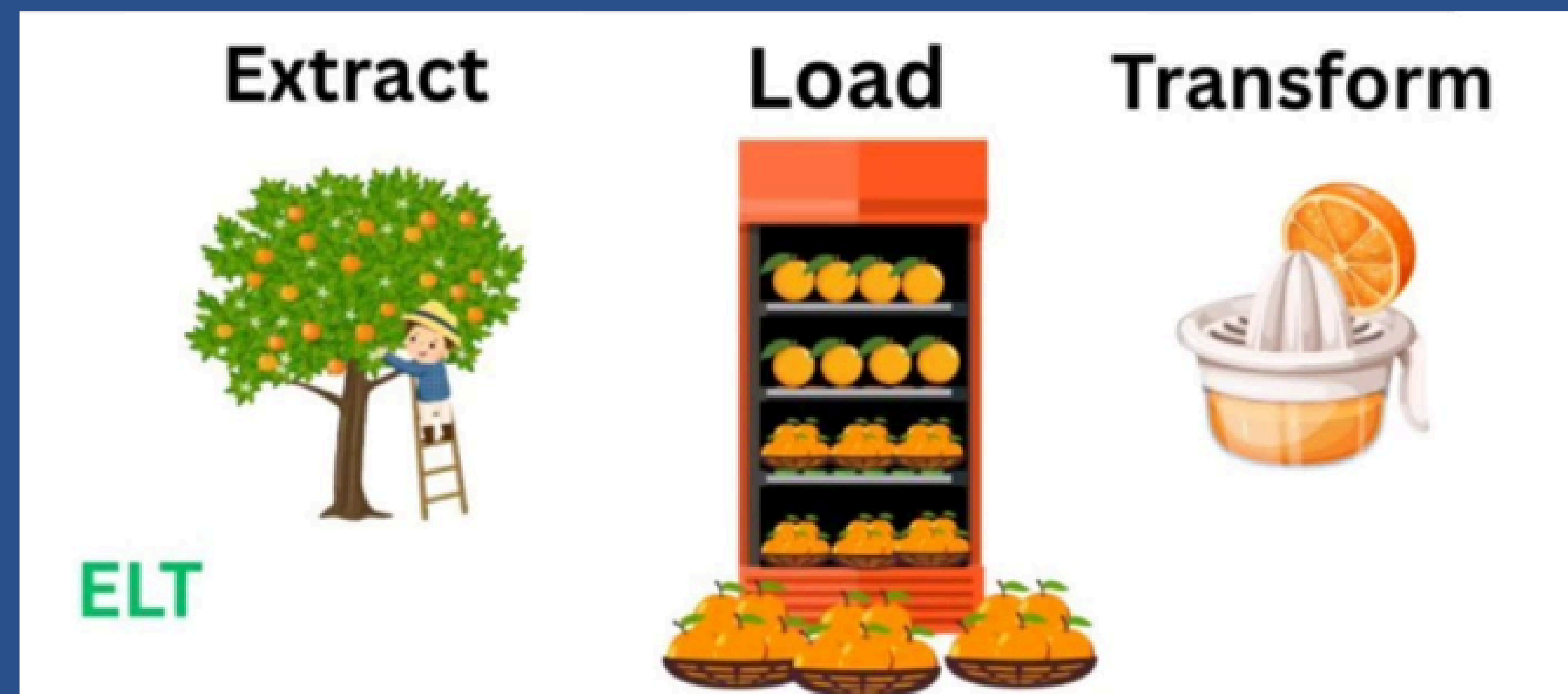
Postman habla HTTP, es el código de nuestro servidor (la API en Node, Python, Java) el responsable de traducir esa petición HTTP al lenguaje nativo de nuestra base de datos (SQL, BSON, Cypher, etc.).

Data Warehouse vs. Data Lake



- Son arquitecturas de datos distintas para diferentes propósitos.
- El Data Lake es ideal para almacenar grandes volúmenes de datos sin procesar y para realizar análisis avanzados.
- El Data Warehouse se utiliza para la toma de decisiones empresariales y análisis estructurados.

ETL vs. ELT



Ambos procesos describen las tuberías de datos (pipelines) que transportan la información desde los sistemas operativos hasta los analíticos, pero cambian el orden de los factores.

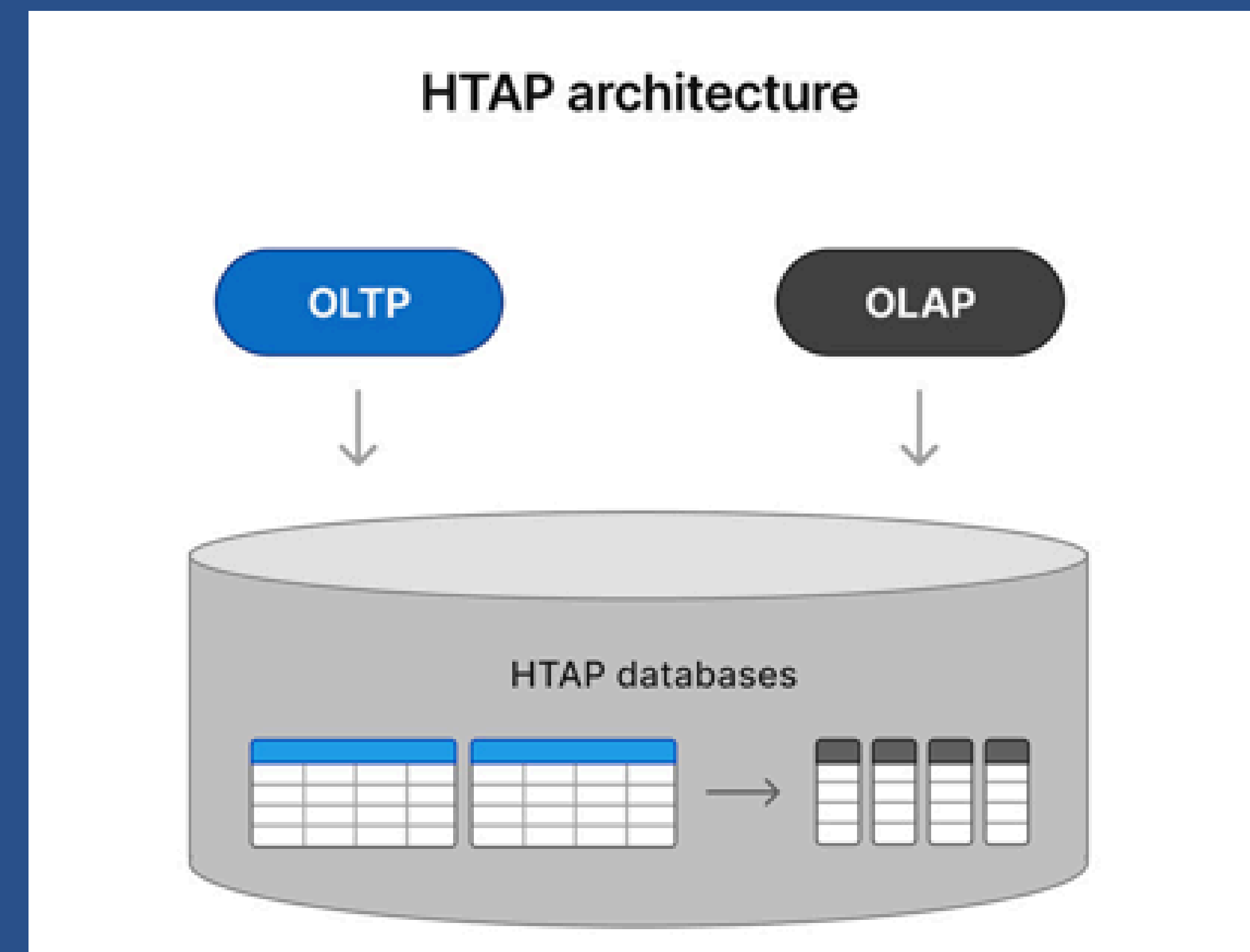
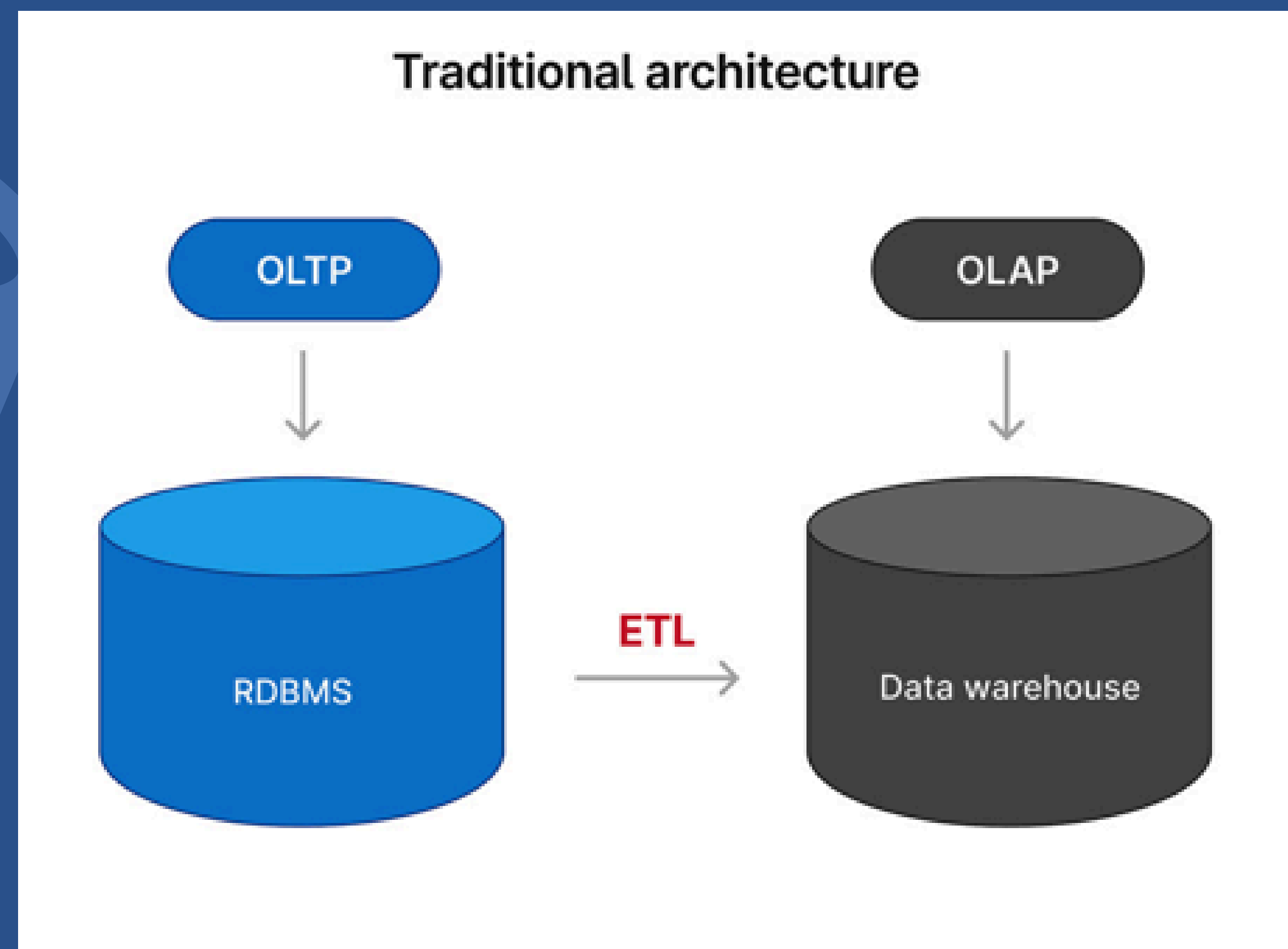
OLTP y OLAP

- OLTP se enfoca en gestionar transacciones diarias en tiempo real (operar el negocio).
- OLAP se dedica al análisis de grandes volúmenes de datos históricos (entender el negocio).
- Al ser procesos con diferente enfoque, utilizar un mismo sistema para ambas tareas degradaba el rendimiento.



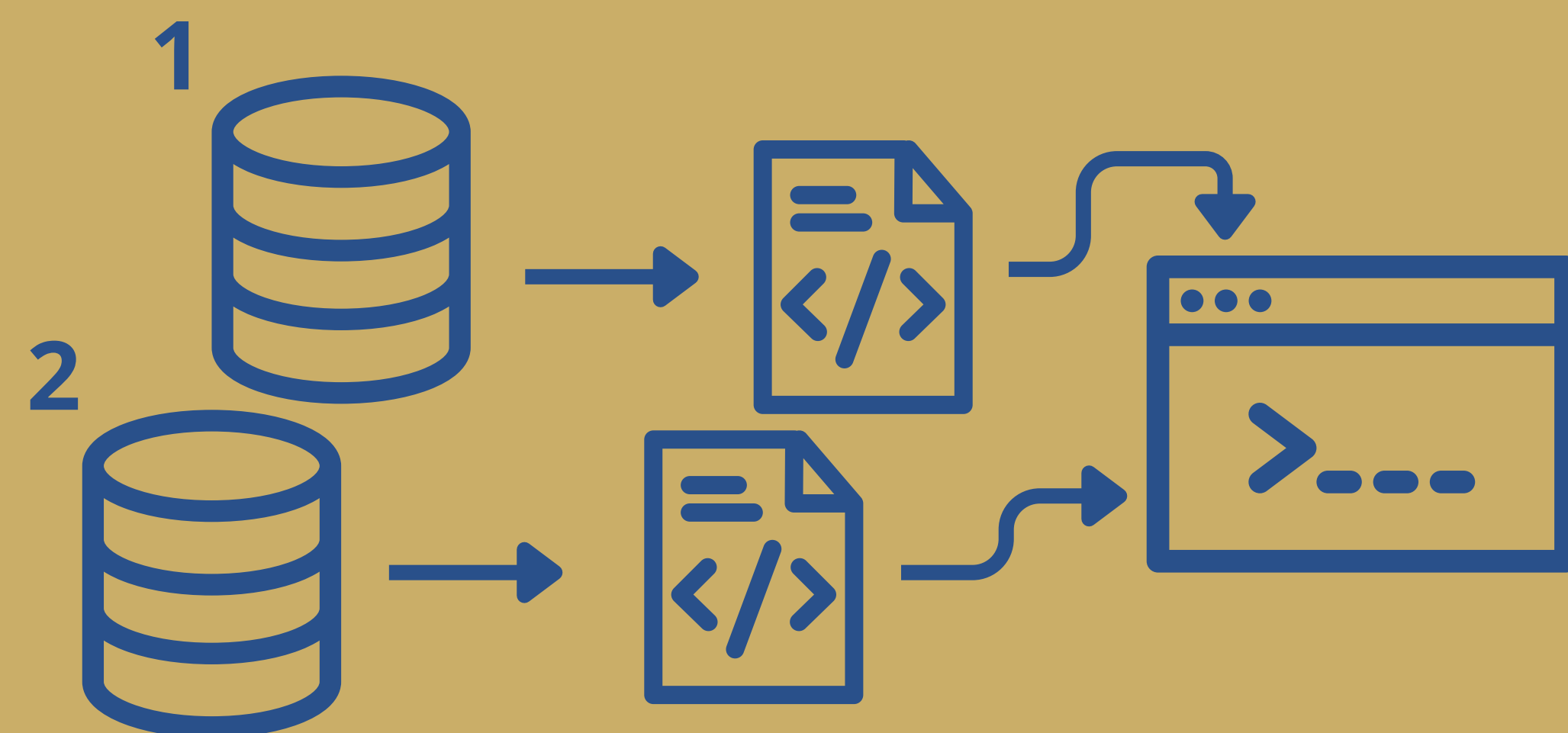
HTAP

Las plataformas HTAP están diseñadas para ejecutar ambos procesos de forma simultánea. HTAP elimina esta espera utilizando tecnologías avanzadas como el almacenamiento en memoria (In-Memory) o copias en espejo optimizadas para diferentes tareas.

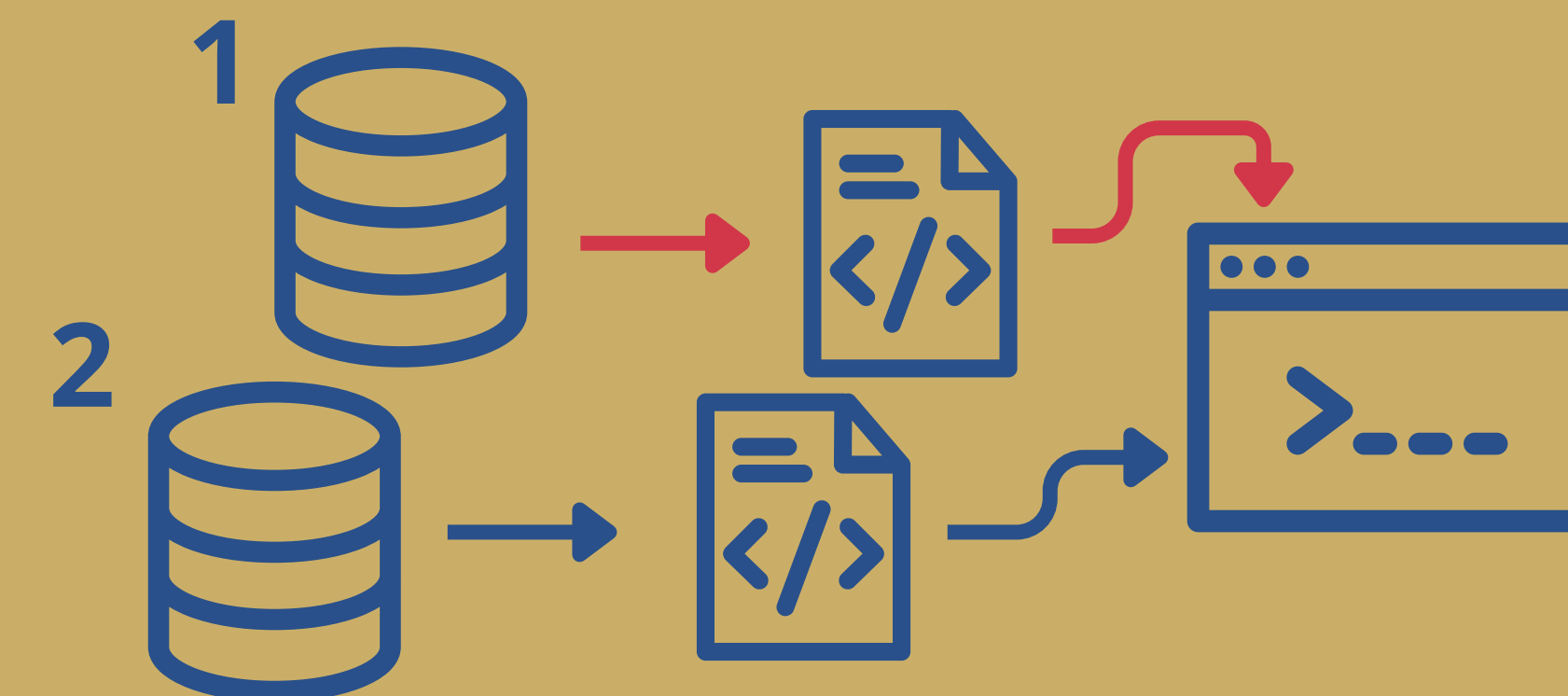


Patrón Saga

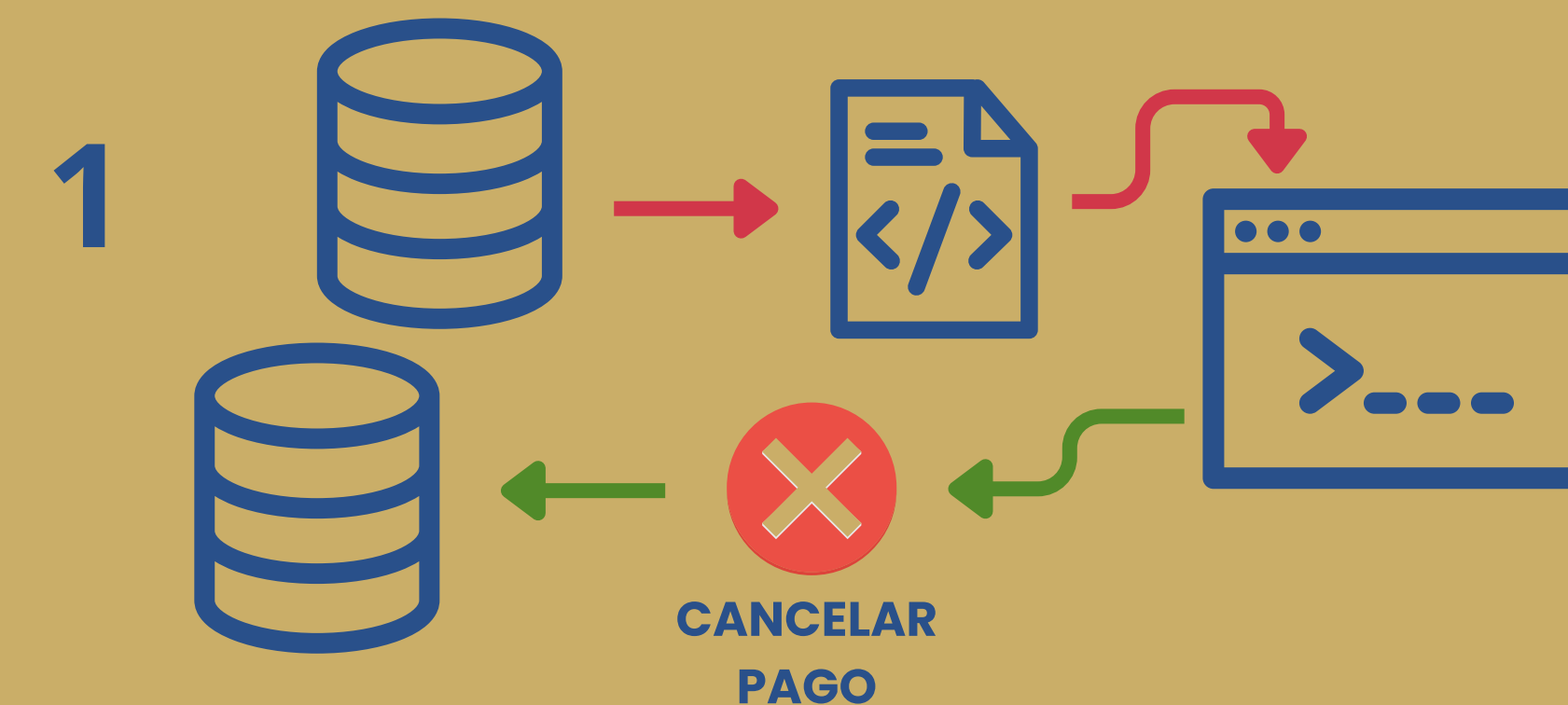
Es una secuencia de transacciones locales coordinadas, cuyo objetivo es disparar mecanismos automáticos si el proceso falla.



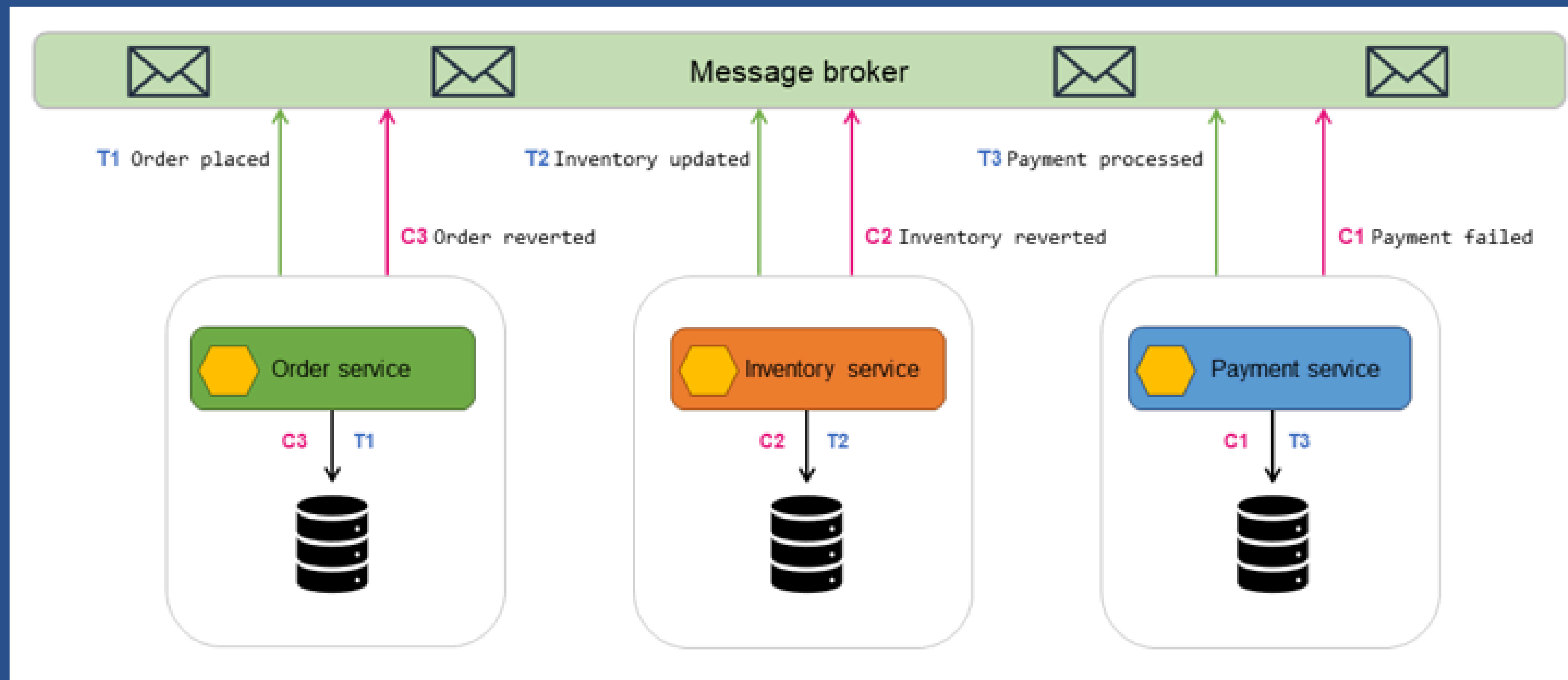
Cada servicio realiza su propia transacción en su propia base de datos y luego publica un evento para desencadenar el siguiente paso.



Si un paso falla, el sistema ejecuta Transacciones Compensatorias



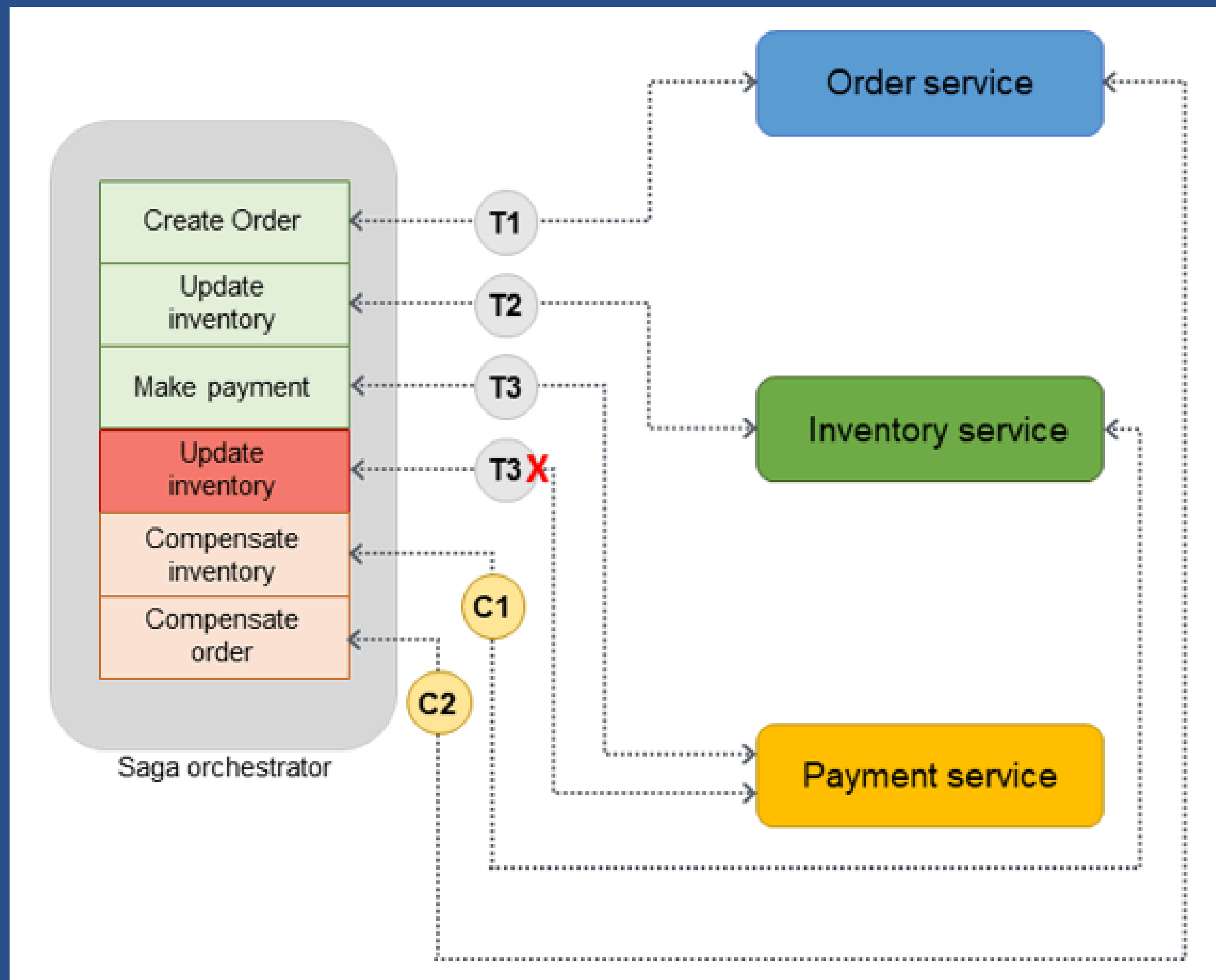
Implementación de Saga: Coreografía



Cada base de datos reacciona a los eventos de las demás de forma autónoma. Es ideal para flujos cortos y muy desacoplados, pero difícil de rastrear si la arquitectura crece demasiado.

Para que funcione, se crea el código del saga en nuestro backend, y se comunican a través de un Broker de Mensajes (como Apache Kafka o RabbitMQ).

Implementación de Saga: Orquestación



Un controlador central envía comandos específicos a cada base de datos. Tiene control absoluto sobre qué transacción compensatoria disparar, pero introduce un punto de fallo lógico central.

Funciona a través de un Motor de Flujos de Trabajo (Workflow Engine) especializado, que se encarga exclusivamente de dirigir el tráfico y el estado de la operación.

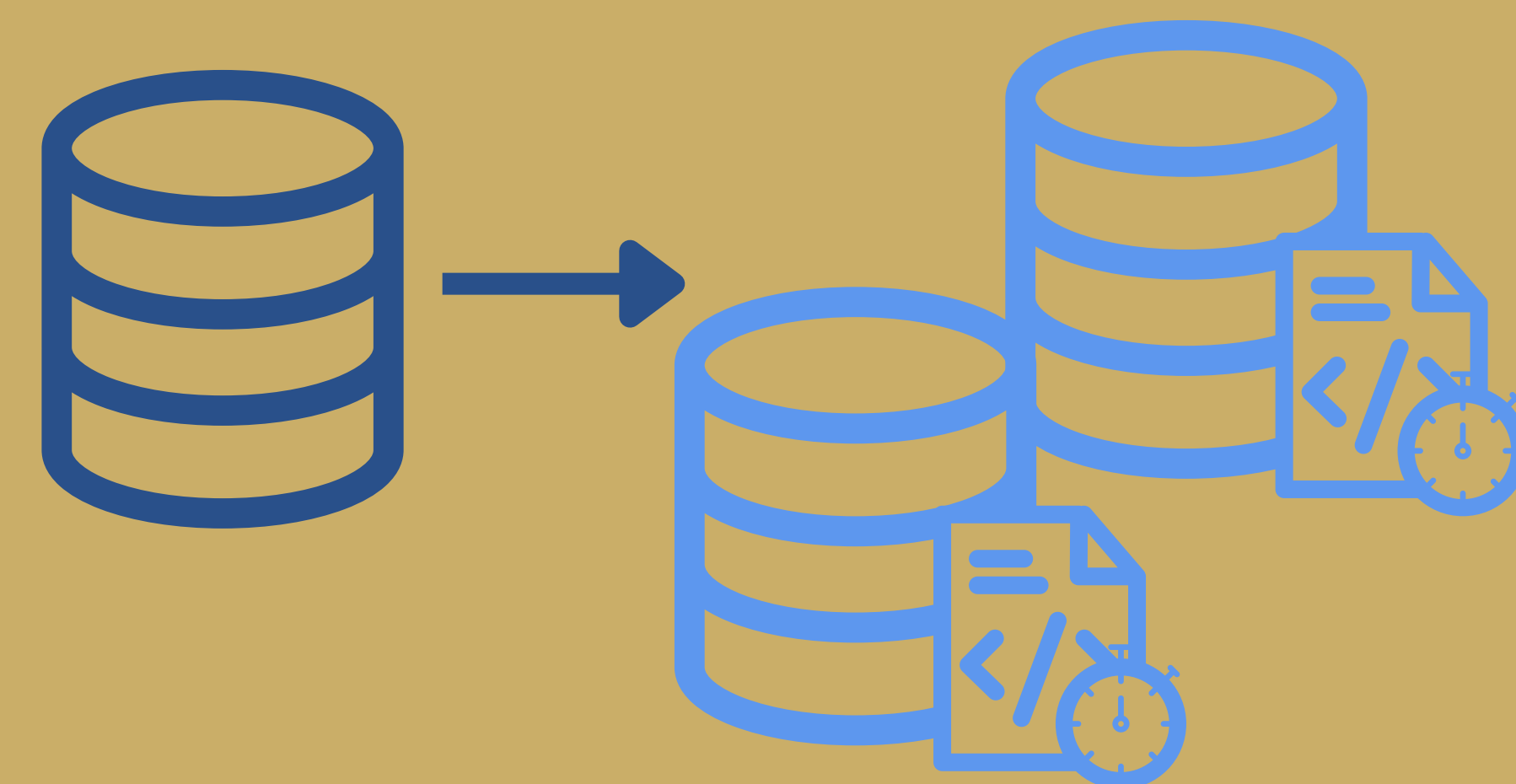
| Migración vs Sincronización

MIGRACIÓN



- **Carácter:** Transitorio con un fin absoluto.
- Mueve datos desde un origen (legacy), a un destino (target).
- Implica transformación y/o desactivación de la Base de Datos de Origen.
- **Fase crítica:** Perfilado y limpieza de Datos.

SINCRONIZACIÓN



- **Carácter:** Permanente y continuo.
- Garantiza que dos almacenes independientes mantengan coherencia o consistencia idéntica en el tiempo.
- Es un **desafío** evitar el **Replication Lag** (retraso).

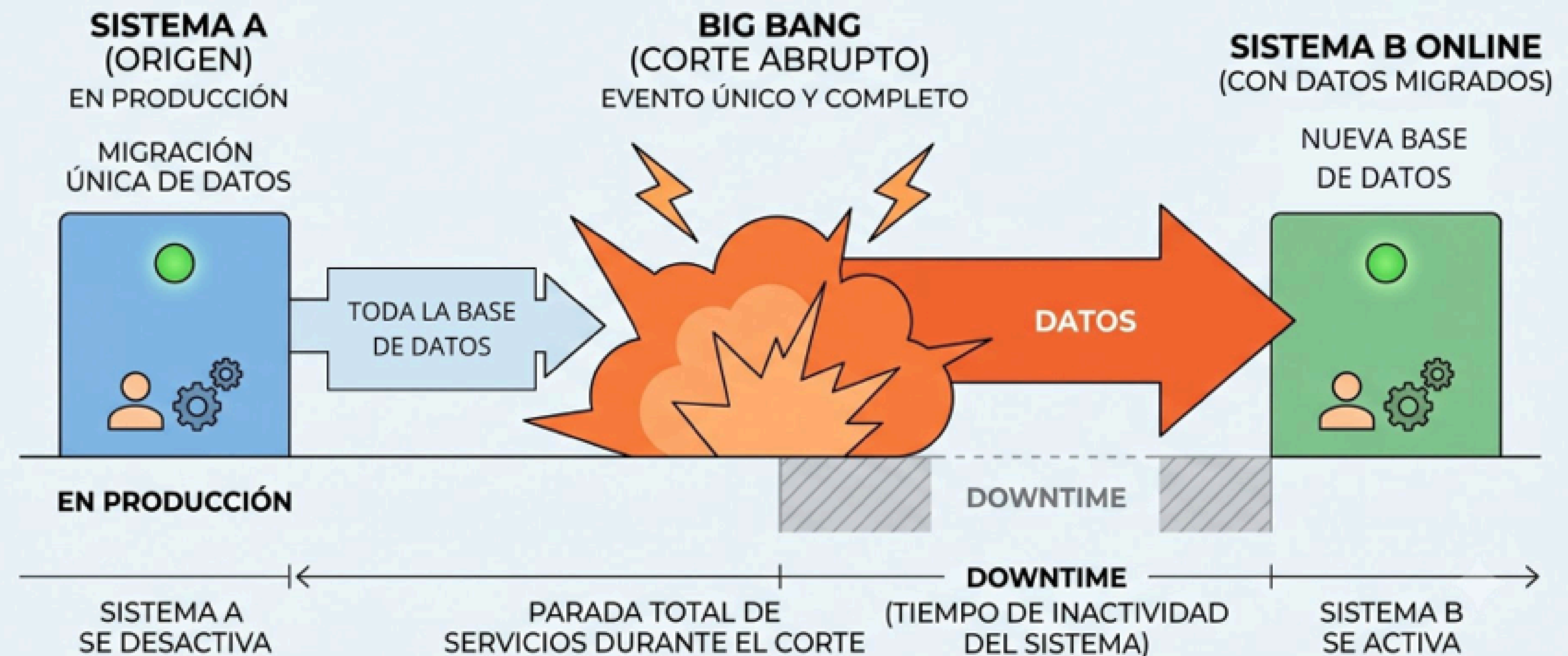
Fases de un Proyecto de Migración



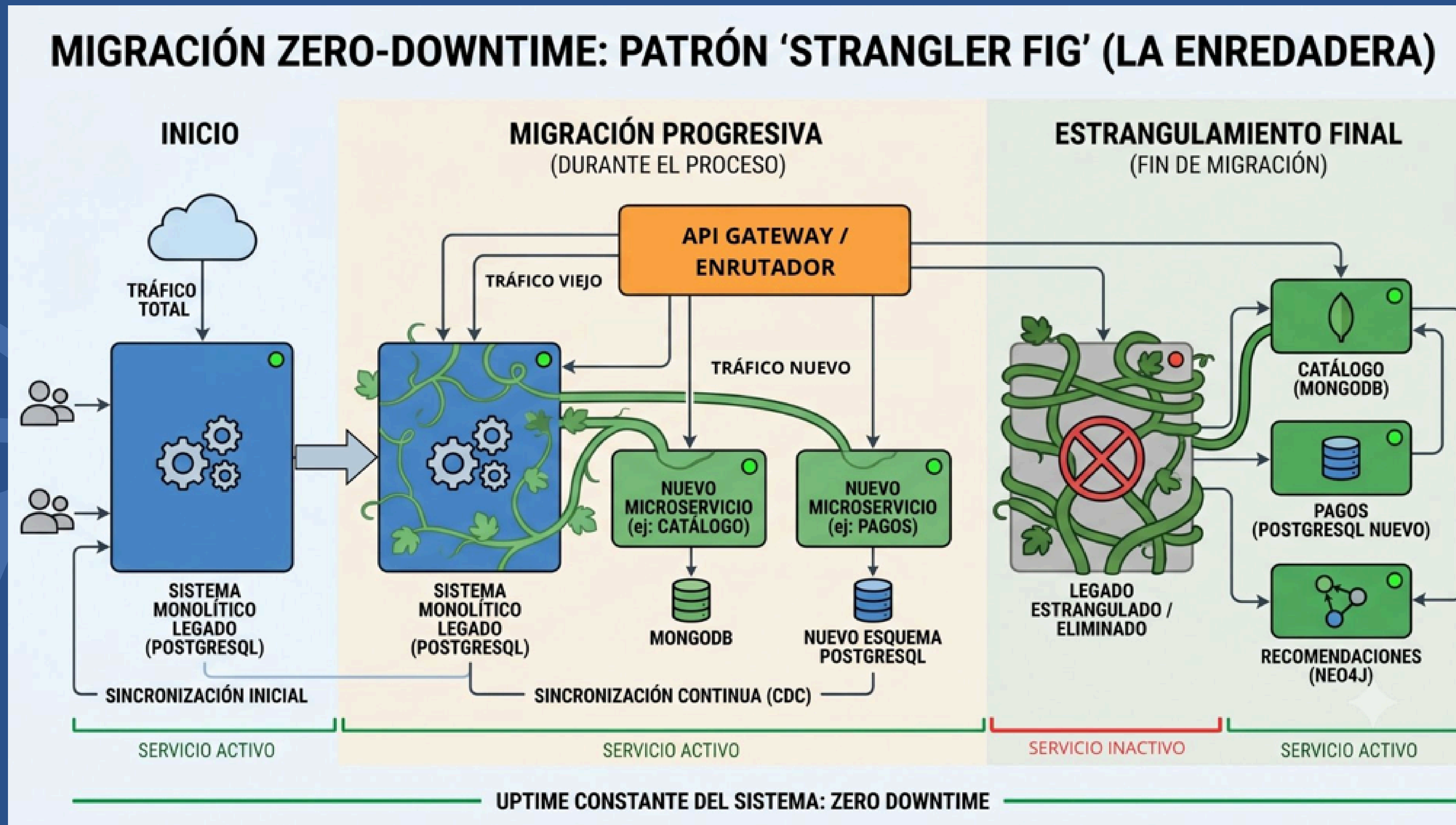
Estrategias de migración: Big Bang

- Corte abrupto que detiene el OLTP.
- Riesgo Alto: Un fallo al 95% requiere un Rollback masivo y Downtime prolongado.

ESTRATEGIA DE MIGRACIÓN: **BIG BANG** (CORTE ABRUPTO)



Estrategias de migración: Zero-Downtime + Strangler Fig



- Migración incremental por microservicios
- Uso de API Gateways para desvío progresivo de lecturas y escrituras.
- Cero Downtime

Estrategias de migración: Trickle

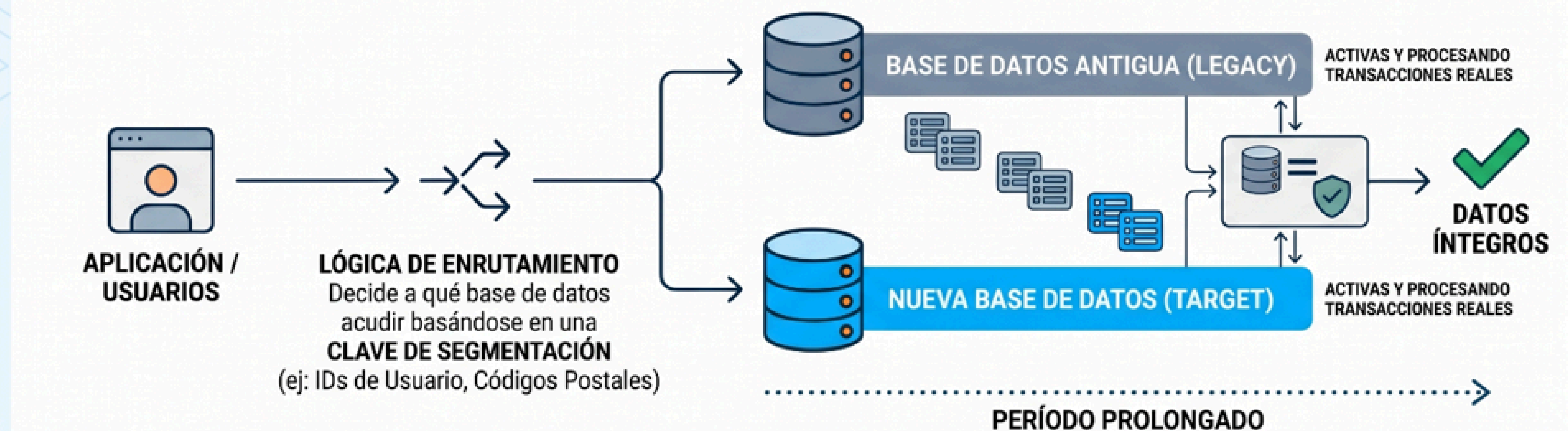
- Ideal para sistemas multi-tenant masivos.

Pasos:

- 1: Segmentación (Data Chunking) por bloques.
- 2: Enrutamiento dinámico basado en clave (Router inteligente).
- 3: Doble escritura cruzada asincrónica para seguridad.
- 4: Lazo de conciliación para reparar discrepancias

ENFOQUE C: MIGRACIÓN TRICKLE (POR GOTEO / FASES EN PARALELO)

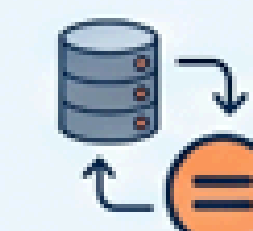
Estrategia de Mayor Seguridad Operativa para Migración de Datos a Gran Escala



DIVISIÓN POR SUBCONJUNTOS:
Se migran bloques de registros (ej: Usuarios 1-1000, luego 1001-2000), no componentes completos.



EJECUCIÓN EN PARALELO (PARALLEL RUN): Ambas bases de datos están completamente vivas y procesando transacciones reales simultáneamente.



LAZO DE CONCILIACIÓN (Validación e Integridad):
Proceso continuo que compara y valida la consistencia de datos entre la Base Antigua y la Nueva durante el período de migración. Asegura la integridad y la ausencia de discrepancias.

| Arquitectura CDC

Change Data Capture: el CDC "escucha" los cambios directamente en el archivo de transacciones de la base de datos principal, sin afectar el rendimiento, y generando un evento asíncrono.

COMPONENTES

Source Connector: Debezium lee el WAL/Oplog y envía a Kafka.

Sink Connector: inyecta el mensaje en el destino (Mongo/Redis/Neo4j)

Snapshotting: La foto inicial del pasado para el "Día Cero".

Schema Registry: Gobierna la evolución de las columnas

Mecánicas de Persistencia y Logs

Motor	Mecanismo de durabilidad	Uso en CDC/Integridad
PostgreSQL	WAL (Write-Ahead Log)	Logical Decoding Slots para streaming de eventos JSON
MongoDB	Oplog (Operations Log)	Change Streams exponen reactivamente mutaciones BSON.
Redis	AOF (Append-Only File)	Redis Streams (XADD) con Consumer Groups para persistencia
Neo4j	Tx Log (Transaction Log)	Neo4j Streams captura COMMITS de grafos complejos.

Teorema CAP

Podemos optar entre:

- Sistemas CP: Consistentes.
- Sistemas AP: Disponibles

